LAYER 9 OF 13

# RATE LIMITING

*Keeping Your API Bill from Exploding*

## TIER 1 — SOLOPRENEUR

---

**MATT MURPHY .AI**

mattmurphy.ai

Part of the Builder Certification Series

# What This Kit Covers

What This Kit Covers

This kit covers Layer 9: Rate Limiting—putting a speed limit on how many requests your app can make and receive. Every time your app does something—loads a page, sends a notification, checks a user's status, calls an AI model—it makes a request. And many of those requests cost money. Rate limiting is the system that controls how many requests happen in a given time period, so your costs stay predictable and your app stays online.

Here's a real scenario: you build a search feature that calls an AI API every time a user types a letter. One user typing a 50-character search query just triggered 50 API calls in ten seconds. Multiply that by 100 users, and you've just burned through your monthly API budget in an afternoon. Rate limiting prevents this by saying "each user gets a maximum of 10 requests per minute on this feature." You describe these limits to your AI coding tool, and it builds the throttling logic. Your job is knowing where limits are needed and what numbers make sense.

**This kit also covers related topics:** setting up billing alerts so you get warned before costs spike, managing API keys so you can track which services are making the most calls, monitoring usage so you can spot problems early, and protecting your app from abuse—whether that's a malicious bot hammering your endpoints or a well-meaning user accidentally triggering a runaway loop.

Why It Matters

Without rate limiting, one bad actor or one coding mistake can run up a massive bill overnight. This happens more than you'd think. A vibecoder ships a feature with an API call inside a loop, a user triggers it, and by morning the API provider has charged hundreds or thousands of dollars. Some API providers will happily let you keep calling—and keep charging—until you hit their maximum. Your app won't crash. Your credit card will just keep getting charged.

Rate limiting also protects your app's availability. If your server gets flooded with requests—whether from a bot attack, a viral moment, or a buggy feature—it can slow down or crash for everyone. Rate limiting acts as a pressure valve: it lets a reasonable number of requests through and politely tells the rest to wait. This keeps your app running for legitimate users even when something unexpected happens.

Certification Tier Map

There are three certification tiers. Each one builds on the one before it. Here’s a quick look at who each tier is for and what you’ll prove you can do.

Tier

Who It’s For

What You Prove

Tier 1

Solo builders shipping apps that use paid APIs

You can set basic rate limits on your API endpoints, configure billing alerts, and prevent a single user or bug from generating an unexpected bill.

Tier 2

Growing apps with multiple API integrations and increasing traffic

You can implement tiered rate limits, monitor usage across services, manage API keys with different quotas, and build cost forecasting into your workflow.

Tier 3

Platforms with high traffic, multiple teams, and strict cost governance

You can architect rate limiting at scale—distributed throttling, usage-based billing, abuse detection, and automated cost controls across an entire platform.

---

**TIER 1 CERTIFICATION GOAL**

*You can add basic rate limits to your app's API endpoints, set up billing alerts for paid services, and prevent a single user or coding bug from generating an unexpected bill—all by directing your AI coding tool.*

## What You Need to Know

You don't need to become a systems engineer. You need to understand a handful of concepts so you can tell AI where to add rate limits and set sensible thresholds.

**What a rate limit actually is:** A rate limit is a rule that says "this endpoint can only be called X times per Y time period." For example: "each user can make 60 API calls per minute" or "this search endpoint allows 10 requests per second total." When someone exceeds the limit, they get a temporary error message telling them to slow down. The technical term for that response is a 429 status code—"Too Many Requests."

**Where rate limits are needed:** Not every part of your app needs a rate limit. Focus on the expensive parts: any endpoint that calls a paid API (like an AI model, a maps service, or a payment processor), login and signup forms (to prevent brute-force attacks), and any feature that sends emails or notifications. If a request costs you money or could be abused, it needs a limit.

**Billing alerts (your financial safety net):** Most API providers—OpenAI, Google Cloud, AWS, Stripe—let you set spending alerts. A billing alert is an email or notification that fires when your usage hits a threshold you set. For example: "email me when my OpenAI spending reaches $50 this month." Some providers also let you set hard caps that stop all API calls when you hit a limit. Set both. The alert warns you; the cap saves you.

API keys (tracking who's calling what): An API key is like an ID badge for your app. When your app calls an external service, it presents its API key to prove it has permission. Each key can have its own limits and permissions. Never share API keys publicly, and use different keys for development vs. production so a testing mistake doesn't eat into your real budget.

Throttling vs. blocking: Throttling slows down requests—"you can keep going, but only at this pace." Blocking stops them entirely—"you've hit the limit, try again in 60 seconds." Most rate limiting starts with throttling and escalates to blocking. Your AI tool can implement either approach; you just need to decide which behavior makes sense for each part of your app.

## Your Toolkit

Your AI coding tool builds the rate limiting logic. These are the services and tools that make it easier to set up and monitor.

API provider dashboards (OpenAI, Vercel, Supabase): Every major API provider has a usage dashboard that shows how many calls you've made, what they cost, and where you are relative to your limits. Check these weekly at minimum.

**Upstash or Redis:** Tools for storing rate limit counters. When your app needs to remember "this user has made 45 requests in the last minute," it stores that count in a fast database like Redis. Upstash is a managed version that works well with serverless apps.

**Billing alert systems:** Set alerts on every paid API you use. OpenAI, AWS, Google Cloud, and Vercel all have built-in alert settings. Configure them before you launch, not after you get the bill.

## Certification Exam Topics

Every exam question is scenario-based. You'll see a real situation and need to identify what's right, what's wrong, or what to do next.

**Runaway costs:** Your app calls an AI API every time a user types in a search box. After a weekend, you get a $400 bill. What caused this, and what rate limiting strategy would prevent it?

429 responses: A user reports getting a "Too Many Requests" error when using your app normally. How do you investigate whether your rate limits are too strict?

**Billing alerts:** You're launching an app that uses three paid APIs. What billing safeguards should you set up before launch day?

**API key security:** Your API key for a paid service is committed to your public GitHub repository. What's the immediate risk, and what do you do?

**Login protection:** Your app's login form has no rate limiting. Why is this a problem, and what limit would you set?

Throttle vs. block: Your app has a "send message" feature. Should you throttle or block users who exceed the limit? What's the trade-off?

**Cost per request:** You're evaluating two AI APIs for a feature. API A costs $0.01 per call. API B costs $0.001 per call but has lower quality. Your feature gets 10,000 calls per day. How do you think about this decision?

**Monitoring:** How do you know if your rate limits are working? What metrics should you be checking after launch?

# Common Pitfalls

These are the rate limiting mistakes vibecoders make most often. They're expensive mistakes—sometimes literally. If you recognize any of these in your own app, fix them before sitting for the exam.

Not setting any rate limits because "my app doesn't have many users yet." One bot, one viral moment, or one coding bug can generate thousands of requests in minutes. Rate limits are cheap to set up and expensive to skip.

Setting up API calls without billing alerts. Every paid API should have a spending alert configured before you write a single line of code. This is your financial safety net—without it, you're flying blind.

Calling an API on every keystroke (like autocomplete search) without debouncing. Debouncing means waiting until the user stops typing before making the API call. Without it, a 20-character search term triggers 20 API calls instead of one.

Using the same API key for development and production. A testing bug in development can eat through your production API budget. Use separate keys with separate limits.

Not implementing retry logic with backoff. When an API returns a rate limit error, your app should wait and try again—not immediately retry in a tight loop, which just makes the problem worse and can get your account suspended.

Ignoring the 429 status code in API responses. A 429 means "Too Many Requests"—the API is telling you to slow down. If your app doesn't handle this gracefully, users see ugly error messages or broken features.

---

# Tier 1 Self-Assessment Checklist

Before you take the exam, run through these questions. Every "no" is something to work on.

Have you set up billing alerts on every paid API your app uses?

Do your most expensive or most-called endpoints have rate limits in place?

Can you check your API usage dashboards and explain what you're spending money on?

Does your app handle 429 (Too Many Requests) responses gracefully instead of showing users an error?

Are you using separate API keys for development and production?

Have you implemented debouncing on any feature that calls an API based on user input (like search)?

Do you know the per-request cost of each paid API your app uses?

# AI Audit Prompt Template

```
Copy this prompt into your AI coding tool to get a quick rate limiting health check on your app. It checks the same things the certification exam covers.
Review my app's rate limiting and cost management setup. For each item, tell me pass or fail with a specific example:
Rate limits on expensive endpoints: Are the endpoints that call paid APIs protected with rate limits? What are the current thresholds?
Billing alerts: Are spending alerts configured on all paid API providers? What are the alert thresholds?
Debouncing: Are API calls triggered by user input (search, autocomplete, filtering) debounced so they don't fire on every keystroke?
429 handling: Does the app handle "Too Many Requests" responses gracefully—with retry logic, not just error messages?
API key management: Are API keys stored securely and are separate keys used for development vs. production?
Usage monitoring: Can you see API usage trends and cost data in a dashboard or logging system?
Cost per feature: Can you estimate how much each API-dependent feature costs per user per month?
Give me an overall score out of 7 and list the top 3 cost risks to address first.
What’s Next
Once you've gone through this kit and can answer "yes" to the self-assessment checklist, you're ready for the Layer 9 certification exam at your target tier.
The best way to prepare: check your actual API spending right now. Log into every API provider you use and look at your usage dashboards. Set up billing alerts if you haven't already. Then review your app's code and find every place it makes an API call—ask your AI tool to add rate limits and debouncing where needed. Real cost management is the proof of work the exam tests.
```

---

**CERTIFICATION PATHWAY**

**Associate Builder:** Pass all 13 layers at Tier 1

**Certified Builder:** Pass all 13 layers at Tier 1 + Tier 2

**MADE Certified:** Pass all 39 tier exams + capstone project

Each exam requires 80% to pass. You can retake after a 24-hour cooldown. No rush—take the time to build something real first.

**MATT MURPHY .AI**

mattmurphy.ai